

SINGAPORE-MIT ALLIANCE

Computational Engineering CME5232: Cluster and Grid Computing Technologies for Scientific Computing COMPUTATIONAL LAB No. 2 13th July, 2007

GETTING STARTED ON PARALLEL PROGRAMMING USING MPI AND ESTIMATING PARALLEL PERFORMANCE METRICS

Question 1: Play Ping-Pong Using 2 Processors on the SMA Hydra Cluster

Write a program, using simple send and receive MPI functions, in which 2 processors are sending their process ranks to each other. At the end they both print the values of what they sent as well as what they received. A sample skeleton source code **pingpong.c** or **pingpong.cpp** is provided for your reference.

(a) Required PC Software **X-Win32**

X-Win32 is a Windows X-server for 32-bit computer. X-Win64 is 64-bit version of X-Win32.

(b) Where to download the sample skeleton code?

Download from **/home/cme5232/clab/clab2/qn1/**.

(c) What should you add on to the skeleton code?

Properly insert **MPI_Send()** and **MPI_Recv()** in the program.

(d) How to compile **pingpong.c** or **pingpong.cpp**?

C : mpicc pingpong.c -o pingpong -lm -mpilog

C++: mpicxx pingpong.cpp -o pingpong -lm -mpilog (Note: **mpicxx** can also compile **C** code)

(e) How to run MPI job in SMA Hydra Cluster?

To submit job directly, you need to define your own machine file, i.e. the compute nodes use for computation. See the following sample_machinefile:

```
-----  
node42  
node50  
-----
```

\$ mpirun -np 2 -nolocal -machinefile sample_machinefile pingpong

*You are not supposed to run mpirun in submit node since this will affect the performance of the system greatly
Use SGE to submit job instead*

It is compulsory to submit MPI job via SGE. Below are some commonly used SGE commands

SGE Commands	Functions	Command Usage Examples
qsub	Submit batch jobs (accept only shell scripts, not executables)	qsub <script> qsub -cwd -S /bin/sh <script>
qstat	Retrieve job status information	qstat qstat -f qstat -u user qstat -j <job_id>
qdel	Delete/Cancel job	qdel <job_id>

		qdel -f <job_id>
qmod	Suspend or Resume Job	qmod -s <job_id> qmod -r <job_id>
qmon	Provides an X Windows GUI command interface and monitoring facility	qmon
qconf	Provides the user interface for cluster configuration and queue configuration.	qconf -sql
qghost	Displays status information about execution hosts	qghost

(f) A simple job script example **simple_sge.sh**

\$ vi simple_sge.sh

```
-----
#!/bin/sh
# Above line is the default shell interpret for Sun Grid Engine
#
# Request Bourne shell as the shell for the job
#$ -S /bin/sh
#
# Run in current working directory, i.e. submit directory
#$ -cwd
#
# Print date and time
date
# Print hostname where the job is running
hostname
-----
```

Little explanation about the simple serial job script

#\$ - The prefix to add SGE options to the script file (special comment lines)

- Comment lines. They are not read by shell.

Some commonly used SGE options:

SGE Options	Explanations
-S shell_name	Specify the shell interpreter to be use for the job.
-V	Export all environment variables
-N job_name	Specify the name of the job to be used by SGE. Without this option, the name of the job is the filename of the submit job script.
-cwd>	Run job in current working directory. Without this option, the job's output file (stdout, stderr) will always put in your home directory.
-o filename	Specify the filename where SGE should save the messages from the standard out channel of your job. Without the -o option, the name of your job (usually the name of the job script, if not specified with -N) plus the extension .o## is used, where '##' is the job number assigned by SGE.
-e filename	Specify the filename where SGE should save the messages from the standard error channel of your job. Without the -e option, the name of your job (usually the name of the job script, if not specified with -N) plus the extension .e## is used, where '##' is the job number assigned by SGE.
-j y	Write standard error to the same file as standard output.

(g) Submit **simple_sge.sh** to SGE and monitor the job status

\$ qsub simple_sge.sh

Your job 1 ("simple_sge.sh") has been submitted.

\$ qstat

...

By using **qstat**, any jobs running or pending in the queue will have a job number and a job status.

- **qw** (queue and waiting)
- **t** (job transferring and about to start)
- **r** (job is running on listed hosts)
- **d** (job has been marked for deletion)

(h) Outputs produced by submit **simple_sge.sh** to SGE

When a job is queued, it is allocated a job number. Once it starts to run, output are usually sent to standard output and standard error, and they are spooled to files called

- **<job_name>.o<job_id>** : Standard output of the job
- **<job_name>.e<job_id>** : Standard error of the job

(i) Job submit script for **pingpong** MPI job

\$ vi pingpong_sge.sh

```
-----
#!/bin/sh
#$ -S /bin/sh
#$ -cwd
#$ -V
#$ -N pingpong-check
# pe (parallel environment) request for MPICH and number of CPUs ($NSLOTS)
# This command line is usually used outside the SGE file for convenience.
#$ -pe mpich 2
myjob=pingpong
mpirun -np $NSLOTS -machinefile $TMPDIR/machines $myjob
```

SGE uses the concept of a parallel environment **PE** (**PE**: a list of hosts along with a set number of job slots and PRE/POST execution) to execute MPI jobs. Each host has an associated queue and resources (CPU, memory).

\$ qsub pingpong_sge.sh

Your job 2 ("clab2_qn1-pingpong") has been submitted.

Note: To put the pe (parallel environment) command line outside the SGE file for convenience, we omit this command line in SGE script file and insert it on the submitting job command as follows:

\$ qsub -pe mpich 2 pingpong_sge.sh

In addition to the **<>.o** and **<>.e** files, there are two more outputs files with parallel jobs

- **<job_name>.po<job_id>** : Parallel execution standard output of the job (hostfile list)
- **<job_name>.pe<job_id>** : Parallel execution standard error of the job (hostfile list)

If the job fails for any reasons, it is **<>.o** and **<>.e** one should examine to determine failed reason. The **<>.o** can often be used to check on the progress of the job. Check **<job_name>.o<job_id>** for the output of your **pingpong** program. Check out other SGE output files as well.

(j) How to obtain a graphical view of the message passing between processors?

Use **Jumpshot** - a Java-based visualization tool for doing postmortem performance analysis.

Jumpshot-4 is installed in SMA Hydra cluster. It supports **.slog2** log file format. **mpicc/mpicxx**

with option “-mpilog” will produce **pingpong.clog**. Use **clogTOslog2** to convert **.clog** to **.slog2**. Run the **X-Win32** software on your PC desktop. Copy down the IP address of your PC.

```
$ clogTOslog2 pingpong.clog (Note: Convert .clog to .slog2)
$ export DISPLAY=<IP Address of your PC Desktop>:0 (Note: 0 is display number)
$ jumpshot pingpong.slog2
```

Jumpshot-4. Play around the diagram for diagrams offer a lot of improves features and useful information some time. See <http://www-unix.mcs.anl.gov/perfvis/software/viewers/> for details.

Try **\$ jumpshot pingpong.clog**.

Question 2: Estimate bandwidth and message latency on the SMA Hydra Cluster

Write a short program to estimate the message latency t_s and reciprocal of the bandwidth t_c for the SMA Hydra Cluster. This can be done by repeatedly sending fixed-size messages from one process to another. When the receiving process receives the message, it should reply to the sending process. The ping-pong program developed in Question 1 can be used for this question. This process should then be repeated with messages of different sizes. After collecting the timings, construct a least-squares-fit line to resulting (message size, time) pair. The intercept on the time vs message size curve should give an estimate of the startup cost t_s while the gradient of the slope will give an estimate for t_c .

(a) How many different message sizes to run?

A possible scheme is to start the message size with 100 bytes, increase by a step size of 100 bytes until the message size reaches 10000 bytes. These will be a total of 100 different message sizes and should be enough to construct a least-square-fit line to the resulting (message size, time) pair.

(b) Proper timing measurement

Properly place `MPI_Wtime()` in the program to obtain more accurate timing.

(c) How to calculate Round Trip Time (RTT) for each message size more accurately?

RTT is the total time taken in completing a “pingpong” process. To obtain a more accurate RTT, you might want to obtain several RTTs for each message size and then take the average value.

(d) How to construct a least-square-fit line to the resulting (message size, time) pair?

You can use Microsoft Excel to draw out the (message size, time) pair and construct the least-square-fit line. Note that the value of the “time” in (message size, time) pair is $RTT/2$. The reason is because we are sending the message twice during the “pingpong” process.

Question 3: Exercises based on sample MPI codes

(3a) Compile and run the parallel integration code **integral.c** using different number of processes. Use the **Jumpshot** facility in MPICH to display the space-time status of your parallel computation. What happens if you try to run it with just one processor? Run the sequential code **ori_integral.c** on one processor. What are the differences between running a sequential code and a parallelized code using one processor?

(a) What code has been added in **integral.c**?

- Parallelized code that performs integration
- MPE code that performs customized logging

(b) Parallelized code that performs integration

The following is the portion of the code performing integration.

```
-----
h = (b - a) / n; /* h is the same for all processes */
local_n = n / p; /* So is the number of trapezoids */

local_a = a + my_rank * local_n * h;
local_b = local_a + local_n * h;
integral = Trap(local_a, local_b, local_n, h);

/* Add up the integrals calculated by each process */
if (my_rank == 0)
{
    total_integral = integral;
    for (source=1; source<p; source++)
    {
        MPI_Recv(&integral, 1, MPI_DOUBLE, source, tag, MPI_COMM_WORLD, &status);
        total_integral = total_integral + integral;
    }
}
else MPI_Send(&integral, 1, MPI_DOUBLE, dest, tag, MPI_COMM_WORLD);
-----
```

(c) MPE code that performs customized logging

The **MPE** (Multi-Processing Environment) attempts to provide a complete suite of performance analysis tools for their MPI programs based on post processing approach. These tools include a set of profiling libraries, a set of utility programs, and a set of graphical visualization tools. The profiling libraries provide a collection of routines that can create log files. These log files can be created manually by inserting MPE calls in the MPI program, or automatically by linking with the appropriate MPE libraries, or by combining the above two methods. Currently, MPE offers the following 3 libraries:

- (i) Tracing library: This traces all MPI calls. Each MPI call is preceded by a line that contains the rank in **MPI_COMM_WORLD** of the calling process, and followed by another line indicating that the call has completed. Most send and receive routines also indicate the values of count, tag, and partner (destination for sends, source for receives). Output is to standard output. (**-o mpitrace** to compile and link with tracing library)
- (ii) Animation library: It does a simple form of real-time program animation. (**-o mpianim** to compile and link with animation library)
- (iii) Logging library: They form the basis to generate log files from user MPI programs. It is the most useful and widely used profiling libraries in MPE. (**-o mpilog** to compile and link with logging library)

In this course, the logging library is the preferred profiling library whenever profiling is required. There are currently 3 different log file formats allowed in MPE. The default log file format is **clog** which is basically a collection of events with single timestamps. **alog** is provided for backward compatibility reason. The most powerful one is **slog2** (Scalable LOGfile, it was .slog previously), which can be converted from **clog** after **clog** file has been generated (preferred approach), or can be generated directly when MPI program is executed (through setting the environmental variable **MPE_LOG_FORMAT** to **SLOG**). The set of utility programs in MPE includes **clogTOslog2**, **logviewer**, and so on. Currently, MPE's graphical tools include 2 display programs, **Upshot** for **.alog** and **Jumpshot-4** for **.slog2**.

Customized logging

In addition to using the predefined MPE logging libraries to log all MPI calls, MPE logging calls can be inserted into user's MPI program to define and log states. These states are called user defined states. States may be nested, allowing one to define state describing a user routine that contains several MPI calls, and display both the user-defined state and the MPI operations contained within it. The routine **MPE_Log_get_event_number()** should be used to get unique event numbers from the MPE system. The routines **MPE_Describe_state()** and **MPE_Log_event()** are then used to describe user-defined states. The following example illustrates the use of these routines.

```
-----
#include "mpe.h"
...
int eventID_begin, eventID_end;
...
eventID_begin = MPE_Log_get_event_number();
eventID_end = MPE_Log_get_event_number();
...
MPE_Describe_state(eventID_begin, eventID_end, "Amult", "bluegreen");
...
MyAmult(matrix m, Vector n)
{
    /* Log the start event along with the size of the matrix */
    MPE_Log_event(eventID_begin, m->n, (char *)0);
    ... Amult code, including MPI calls ...
    MPE_Log_event(eventID_end, 0, (char *)0);
}
-----
```

If the MPE logging library, 'limlmpe.a' is not linked with the user program, **MPE_Init_log()** and **MPE_Finish_log()** must be called before and after all the MPE calls.

The following is the MPE code that performs customized logging in **integral.c**

```
-----
int event1a, event1b;
...
event1a = MPE_Log_get_event_number();
event1b = MPE_Log_get_event_number();
...
MPE_Describe_state(event1a, event1b, "Compute", "red");
...
MPE_Log_event(event1a, 0, "start compute");
/* Code to evaluate the integral within the interval assigned to each process */
MPE_Log_event(event1b, 0, "end compute");
-----
```

(3b) A more accurate alternative to trapezoidal rule is Simpson's rule. It takes the following form in the interval of integration $[a, b]$ which is divided into n even intervals:

$$\int_a^b f(x)dx = \frac{h}{3} [f(x_0) + 4f(x_1) + 2f(x_2) + 4f(x_3) + \dots + 2f(x_{n-2}) + 4f(x_{n-1}) + f(x_n)]$$

Assume that you integrate using even number of processes p so that the ratio n/p is even. Modify the programs **ori_integral.c** and **integral.c** to estimate the value of the integral. State clearly how you have implemented message passing functions and compare the results with those from the trapezoidal rule.

Make sure n/p is even. In order for Simpson's rule to work properly, n/p must be even.

Question 4: Parallelize your sequential code for calculating the value of pi by using the Monte Carlo Method from Computational Lab 1

This question will focus on the parallelization of the sequential code you have written in Computational Lab 1 to evaluate the value of **pi** by using Monte Carol Method.

Write your own parallel codes by using (a) only MPI_Send() and MPI_Recv() (Point-to-Point operations), and (b) Collective operations. Run the parallel codes by varying the number of "darts" thrown at the board N using 2, 4, 8, 16 and 32 processors. Record the execution time (communication time and computation time), the value and precision of your **pi**. Use the **Jumpshot** facility in MPICH to display the space-time status of your parallel computation.

(4a) Compare the value and precision of **pi** obtained from both parallel codes with that obtained from the serial code. (PI25DT = 3.141592653589793238462643 is the true **pi** value up to 25 digits of accuracy)

(4b) Calculate the speedups and efficiencies for both parallel codes. Plot the curves of (i) Predicted and actual Speedup, (ii) Predicted and actual Efficiency for both parallel versions. Comment on your observations.

(4c) Discuss your observations and make statements about the ratio of computation time to communication time. Explain clearly how you have estimated communication and computation time for this problem.

(a) Point-to-Point operations

MPI functions enabling P2P communications involving 2 processes: MPI_Send() and MPI_Recv()

(b) Collective operations

MPI functions involving more than 2 processes

(c) Proper timing measurement

Properly place MPI_Wtime() in the program to obtain a better estimate of communication time and computation time

(d) Calculation of predicted speedup and actual speedup, predicted and actual efficiency

$$\text{Predicted Speedup } S_p = \frac{1}{(1-a) + \frac{a}{P} + \frac{T_D}{T_1}}, \quad \text{Actual Speedup } S_a = \frac{T_1}{T_p}$$

$$\text{Predicted Efficiency } e_p = \frac{S_p}{P}, \quad \text{Actual Efficiency } e_a = \frac{S_a}{P}$$

where P is the number of processors used

a is the fraction of operations that can be parallelized

T_D is the delay due to communications and memory contention

T_1 is the time taken on single processor

T_p is the time taken on p processor